



算法设计与分析

Lecture 1: Introduction

曹刘娟

厦门大学信息学院计算机系

caoliujuan@xmu.edu.cn



WHO AM I?

■ 教师:

- 曹刘娟, 计算机系教授
- 邮箱: caoliujuan@xmu.edu.cn
- 手机: 15060727861
- 助教: 李新阳, 曲延松



群名称: 算法设计与分析2024
群 号: 680237985



课程简介

- “算法设计与分析”是计算机相关专业一门重点专业基础课程，也是学科核心专业基础课程。本课程主要介绍算法设计与分析的基本方法以及算法复杂性理论基础。
- 通过本课程的学习，要求学生理解并熟练掌握递归与分治法、贪心法、动态规划方法、回溯法、分支定界法，以及高级图论算法、线性规划算法等，理解并掌握算法复杂性的分析方法、NP完全性理论基础。
- 通过教学和实践，培养学生从算法的角度运用数学工具分析问题和解决问题的能力，从而使学生能够正确地分析和评价一个算法，进一步设计出真正有效的算法。
- 此外，配合实验课程的教学，学生应理论联系实际，理论指导实践，通过规范地完成一系列算法设计实验进一步巩固所学的相关书本知识，在知识、能力、素质上得到进一步的提高。



教材及参考书

■ 教材：

- 张德富, 算法设计与分析, 国防工业出版社, 2009.

■ 参考书：

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms (the second edition), 算法导论（第二版）机械工业出版社, 2006.
- 屈婉玲等, 算法设计与分析（第二版），清华大学出版社, 2016

■ 参考学习视频

- 北京大学 屈婉玲 算法设计与分析, <https://www.bilibili.com/video/BV1254y1r71T?p=1>

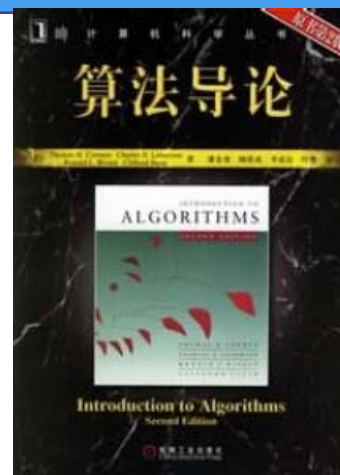
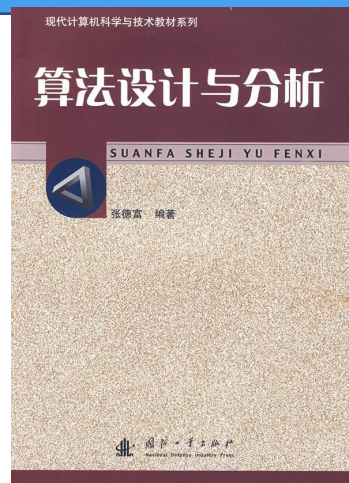


Image source: <http://www.bookschina.com/4203199.htm>
<https://www.amazon.com/Introduction-Algorithms-Second-Thomas-Cormen/dp/0070131511>



日程及教室安排

- 1-14周 星期一 第5节-第6节 学武楼 (1号楼) B511
- 1-14周 星期三 第5节-第6节 学武楼 (1号楼) B511
- 实验上机课: 2-14周(双) 星期二 第1节-第4节 文宣楼 (4号楼) B313,



■ 成绩构成:

- 期末笔试50%
- 实验上机考试15%
- 平时成绩
 - 编程作业（20%）：提交简单报告以及对应的代码
 - 课本习题+出勤+课堂表现（15%）

■ 作业迟交策略:

- 1天内: 扣得分的10%
- 1-3天: 扣得分的30%
- 3天后: 扣得分的50%



- 课件, 资料, 作业发布与递交均在头歌网站上进行.
 - 后续会在QQ群通知
 - 课件以英文为主, 授课语言为中文
- 作业使用中英文均可, 并在该网站上递交电子版 (完全无纸化) .



■ 编程作业将不定期施行代码查重

- 第一次警告
- 第二次该次作业0分
- 第三次...

a.cpp (94%)	b.cpp (94%)
1-17	1-12

a.cpp


```
void insertionSort(int arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;

        /* Move elements of arr[0..i-1], that are
        greater than key, to one position ahead
        of their current position */
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

b.cpp


```
void insert(int A[], int n)
{
    int i, temp, j;
    for (i = 1; i < n; i++) {
        temp = A[i];
        j = i - 1;

        while (j >= 0 && A[j] > temp) {
            A[j + 1] = A[j];
            j = j - 1;
        }
        A[j + 1] = temp;
    }
}
```

非常严格的代码查重系统





算法的应用

- Internet
 - Web search, packet routing, distributed file sharing, ...
- Artificial Intelligence
 - chatGPT聊天机器人, AI画图, AI辅助编程
- Computers
 - Circuit layout, file system, compilers, ...
- Computer graphics
 - Movies, video games, virtual reality, ...
- Security
 - Cell phones, e-commerce, voting machines, ...
- Multimedia
 - MP3, JPG, DivX, HDTV, face recognition, ...
- Social networks
 - Recommendations, news feeds, advertisements, ...
- Physics
 - Aerodynamics simulation, particle collision simulation, ...
- Biology
 - Human genome project, protein folding, ...
- ...



Algorithm application

■ ChatGPT

- $ChatGPT = GPT$ (大规模语言模型) + $RLHF$ (基于人类反馈的强化学习算法)

ME 可以介绍下你自己吗?

当然可以! 我是 OpenAI 的一个大型语言模型, 名字叫做 ChatGPT。我被训练以回答各种问题, 包括但不限于历史、科学、技术、文化和日常生活等方面的问题。我的知识截止日期是 2021 年, 所以有一些新的事物或信息我可能不知道。如果你有什么问题, 可以向我询问, 我会尽我所能帮助你。

write a python program that can count word frequency of a text file and output the result in an ASCII table with sorted Descending

To write a Python program that can count the word frequency of a text file and output the result in an ASCII table with sorted descending, you can use the following steps:

1. Open a new file in your text editor and save it as 'word_count.py'.
2. Import the 'Counter' class from the 'collections' module, and the 'tabulate' function from the 'tabulate' module. These will be used to count the frequency of words in a text and to format the results as an ASCII table.
3. Define a 'count_words' function that takes a string as an argument and returns a dictionary containing the frequency of each word in the string.
4. In the 'count_words' function, use the 'Counter' class to count the frequency of words in the input string.
5. Return the dictionary containing the word frequencies.
6. Use the 'open' function to read the contents of the text file into a string.
7. Call the 'count_words' function and pass the contents of the text file as an argument.
8. Use the 'tabulate' function to format the dictionary of word frequencies as an ASCII table.
9. Use the 'sorted' function to sort the word frequencies in descending order.
10. Print the sorted ASCII table to the console.

Here is an example implementation of the program:

```
from collections import Counter
from tabulate import tabulate

def count_words(text):
    # Use the Counter class to count the frequency of words in the text
    word_counts = Counter(text.split())
    return word_counts

# Read the contents of the text file into a string
with open('text.txt') as f:
    text = f.read()

# Count the frequency of words in the text and format the results as an
# ASCII table
word_counts = count_words(text)
table = tabulate(word_counts.items(), headers=['Word', 'Frequency'],
                 tabfmt='simple')

# Sort the word frequencies in descending order
sorted_table = sorted(table, key=lambda x: x[1], reverse=True)

# Print the sorted ASCII table to the console
print(sorted_table)
```

Create a GPTLang file called 'sort.gptl'. Its function is to sort an array with selection sort algorithms.

Write detailed steps, including creating and compiling the file, and run the result in the terminal.

Here are the detailed steps to create a GPTLang file named 'sort.gptl' that uses the selection sort algorithm to sort an array:

1. Open a text editor or code editor and create a new file named 'sort.gptl'.
2. Write the following code in the 'sort.gptl' file to define the 'selection_sort' function:

```
FUNC selection_sort(array:arr) -> arr:
    # Iterate over the elements of the array
    LOOP array -> elem:
        # Find the minimum value in the remaining unsorted portion of the
        array
        VAR min_index:int = array.index(elem)
        LOOP array[min_index+1:] -> other_elem:
            IF other_elem < array[min_index]:
                min_index = array.index(other_elem)

        # Swap the minimum value with the current element
        array[array.index(elem)] = array[min_index]
        array[min_index] = elem

    # Return the sorted array
    return array
```

3. Save the 'sort.gptl' file.
4. Open a terminal or command prompt and navigate to the directory where the 'sort.gptl' file is saved.



Algorithm application

■ AI画图

- Stable Diffusion, Dalle 2, MidJourney等等





- 又称网页排名，是一种由搜索引擎根据网页之间相互的超链接计算的技术，而作为网页排名的要素之一，以Google公司创办人拉里·佩奇（Larry Page）之姓来命名。



PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites



■ 推荐算法

1. 用户协同过滤 (UserCF) : 相似的用户可能喜欢相同物品。如加了好友的两个用户, 或者点击行为类似的用户被视为相似用户。如两个同学之间各自喜欢的视频, 可能会产生互相推荐。
2. 物品协同过滤 (ItemCF) : 相似的物品可能被同个用户喜欢。如果一个人喜欢吃某个菜的话, 那么他可能喜欢和这个类似的菜。

		1	2	3	4	5	6
A		4	3			5	
B		5		4		4	
C		4		5	3	4	
D			3				5
E			4				4
F				2	4		5

相似
相似度0.27

不相似
相似度0





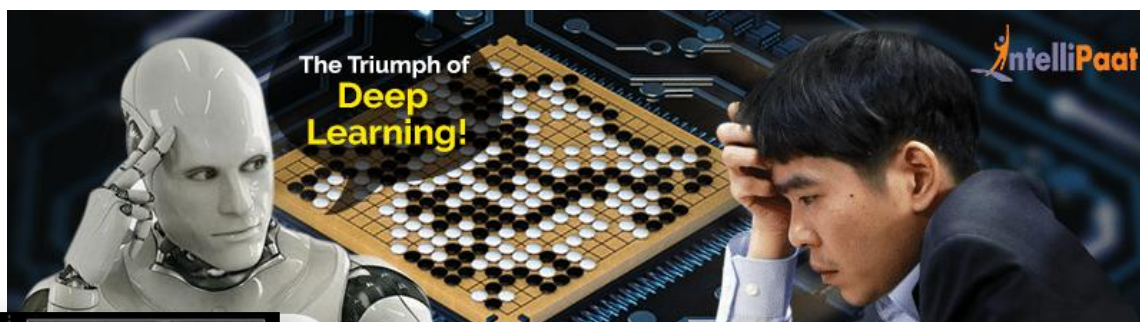
Algorithm application

- 第一个击败人类职业围棋选手、第一个战胜围棋世界冠军的人工智能算法

- AlphaGo中用到的算法



- 强化学习算法 （动态规划算法可以用于强化学习）
- 回溯算法等等





Why algorithm?

- Algorithm engineer is rich!

Job title	Average salary per year in US (2020)
Algorithm Engineer	\$135,272
Database Engineer	\$119,281
Software Engineer	\$108,058
System Engineer	\$102,266
Network Engineer	\$96,220
IT Project Manager	\$95,712
Web Developer	\$75,671

- 保研/考研：笔试和机试大多都会考算法



■ Example: 在手机通讯录里面找“王东”

■ 策略 (Strategy) 1

- 从通讯录记录的第一个人名开始
- 如果第一个人不是王东，就继续找下一个
- 直到找到“王东”

■ 策略(Strategy) 2

- 滑动通讯录，快速定位到姓氏以W开头的人名.
- 检查当前的姓氏是不是“王”
- 找到以“王”开头的人名
- 查找下一个人名，直到找到“王东”

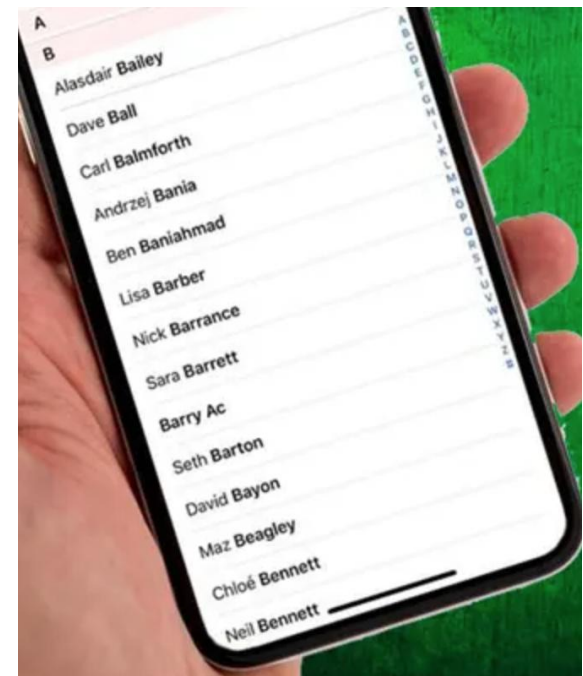


Image source: <https://messagingapplab.com/wp-content/uploads/2019/04/Knowing-Who-Has-My-Number-in-Their-Contact-List.jpg>.webp



什么是算法?

算法是解决问题的一种方法或一个过程，是一个由若干运算或指令组成的有穷序列. 求解问题的算法可以看作是输入实例与输出之间的函数。

$$f(x) \quad x \in S$$

$f(x)$ 目标函数 S 解空间
 $x \in S$ 约束条件

具体到抽象的映射

**为一个实际问题建立
数学模型以方便求解**

- An **algorithm** is a sequence of computational steps that transform the **input** into the **output** to **solve a given problem**.
- Example: The **sorting** (排序问题) problem.
 - **Input**: A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$.
 - **Output**: A permutation (reordering) $A' = \langle a_1', a_2', \dots, a_n' \rangle$ of the input sequence such that $a_1' \leq a_2' \leq \dots \leq a_n'$.



定义

- **Problem (问题):** A question to which we seek an answer.
 - Find the non-decreasing order of a sequence of n numbers A .
- **Algorithm (算法):** A step-by-step procedure applying a technique for solving the problem.
 - Insertion sort
 - Quicksort
 - Mergesort
 - ...
- **Parameters (参数):** Variables that are **not assigned specific values** in the statement of the problem.
 - A .
- **Instance (实例):** Specific assignment of values to the parameters.
 - $A = \langle 3, 6, 1, 7, 2 \rangle$.
- **Solution (答案):** The answer to the problem in that instance.
 - $A' = \langle 1, 2, 3, 6, 7 \rangle$.



算法的性质

- **Finite (有穷性)**: An algorithm consists of finite number of operations.
- **Feasible (可行性)**: Every operation is executable.
- **Deterministic (确定性)**: Every operation must not be ambiguous (generate random results).
- **Input (输入)**: 0 or more, describe the initial state.
- **Output (输出)**: 1 or more, describe the result process from the input.



对算法中步骤的简单语言描述，与特定的编程语言无关

- Pseudocode (伪代码) is a plain language description of the steps in an algorithm, irrelevant to specific programming language.
- We have the following conventions:
 - “ $\leftarrow$ ” represents variable assignment (变量赋值).
 - Indentation (缩进) indicates block structure. “{}” 不是必须
 - While, for, repeat, if, then, and else have the same interpretation as in most programming language.
 - // indicates comments.
 - Composite data type (复合类型) are typically organized into objects, which are comprised of attributes or fields, e.g. T.Lchild.
 - Array elements (数组) are accessed by specifying the array name followed by the index in square brackets, e.g. A[i].
 - Most of the time, we start the index i from 1, instead of 0.



伪代码：插入排序

- Insertion sort (插入排序) is one of the simplest sorting algorithms.
- We use it all the time when we are playing cards.



```
InsertSort( $A$ )  
1  for  $j \leftarrow 2$  to  $n$  do  
2       $key \leftarrow A[j]$   
3       $i \leftarrow j - 1$   
4      while  $i > 0$  and  $A[i] > key$  do  
5           $A[i + 1] \leftarrow A[i]$   
6           $i \leftarrow i - 1$   
7       $A[i + 1] \leftarrow key$   
8  return  $A$ 
```

Image source: 张德富, 算法设计与分析, 国防工业出版社, 2009.



插入排序过程

i j

5	2	4	6	1	3
---	---	---	---	---	---

i j

2	5	4	6	1	3
---	---	---	---	---	---

i j

2	4	5	6	1	3
---	---	---	---	---	---

i j

2	4	5	6	1	3
---	---	---	---	---	---

i j

1	2	4	5	6	3
---	---	---	---	---	---

i $key=2$

2	5	4	6	1	3
---	---	---	---	---	---

i $key=4$

2	4	5	6	1	3
---	---	---	---	---	---

i $key=6$

2	4	5	6	1	3
---	---	---	---	---	---

i $key=1$

1	2	4	5	6	3
---	---	---	---	---	---

i $key=3$

1	2	3	4	5
---	---	---	---	---



■ <https://visualgo.net/zh/sorting>



- 针对数组 $A = \langle 3, 6, 1, 7, 2, 4 \rangle$ 写出插入排序的每个步骤



- 本课程我们将学习以下几类算法
 - Recursive algorithms (递归算法)
 - Divide-and-conquer algorithms (分治算法)
 - Dynamic programming (动态规划)
 - Greedy algorithms (贪心算法)
 - Graph algorithms (图算法)
 - Backtracking (回溯)
 - Branch and bound (分支限界)
 - NP-hard以及NP-complete概念
- They are not specific algorithms, but algorithm families.
- The algorithms in the same family share the same general idea to solve problems.

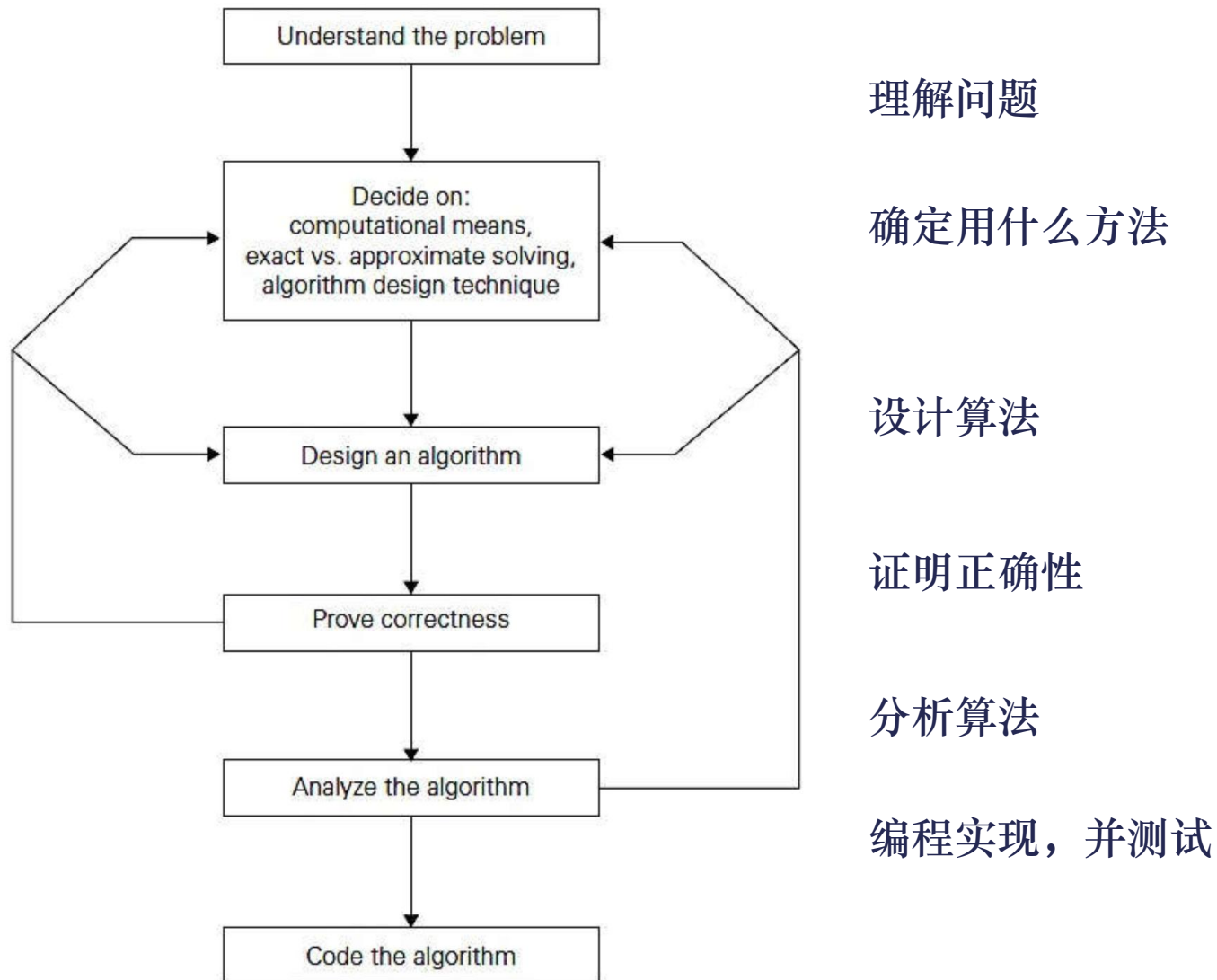


IMPORTANT PROBLEM TYPES

- **Sorting (排序)** : Rearrange the items of a given list in nondecreasing order.
- **Searching (搜索)** : Finding a given value, called a search key, in a given set or multiset.
- **String processing (字符串处理)** : e.g., matching of text strings, find subsequences.
- **Graph problems (图问题)** : e.g., modelling transportation, communication, social and economic networks (graph traversal algorithms, shortest path algorithms).
- **Combinatorial problems (组合优化问题)**: e.g., resource scheduling for wireless communications.
- **Geometric problems (几何问题)** : e.g., closest-pair problem, convex-hull problem for computer graphics and robotic vision.
- **Numerical problems (数值问题)** : Solving equations and systems of equations, computing definite integrals, evaluating functions.



设计和分析算法的过程





- Before we discuss the efficiency of an algorithm, we should make sure one thing: **Correctness (正确性)**.
 - For every input instance, the algorithm halts with the correct output.
- How to prove the correctness? Try every input instance and verify the output?
 - Can we obtain all input instances for the sorting problem?
- 我们必须证明算法的正确性



LOOP INVARIANTS (循环不变量)

- At the start of each iteration of the for loop, the subarray $A[1 \dots j - 1]$ consists of the elements originally in $A[1 \dots j - 1]$ but in sorted order.
- We state these properties of $A[1 \dots j - 1]$ formally as a **loop invariants (循环不变量)**.
- 使用loop invariants 来证明算法的正确性 **why an algorithm is correct.**
- 很多时候采用数学归纳法



■ 第一数学归纳法

- 1.归纳基础 (归纳奠基) : 证明当 $n=1$ 时命题成立;
- 2.归纳假设: 假设当 $n=k$ 时命题成立;
- 3.归纳步骤 (归纳递推) : 由归纳假设推出当 $n=k+1$ 时命题也成立

■ 第二数学归纳法

- 归纳基础: 证明 $n=1$ 时命题成立;
- 归纳假设: 设 $n \leq k$ 时命题成立; (区别在此步)
- 归纳递推: 由归纳假设推出 $n=k+1$ 时命题也成立.



插入排序算法的正确性

The loop invariants are:

$A[1 \dots j - 1]$ is sorted before each iteration.

The proof is similar to mathematical induction (数学归纳法):

- Initiation 初始化 (归纳基础): It is true prior to the first iteration of the loop.
- Maintenance (归纳步骤): If it is true before an iteration of the loop, it remains true before the next iteration.
- Termination(终止): When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

数学归纳法通常针对无穷序列, 但是我们的循环有终止, 所以还多了一个终止



Initialization (初始化/归纳基础)

- We start by showing that the loop invariant holds before the first loop iteration, when $j = 2$.
- The subarray $A[1 \dots j - 1]$, therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$.
- Obviously, the subarray with only one element is sorted, which shows that the **loop invariant** holds at the start of the first iteration of the loop.



Maintenance (归纳步骤)

- At the start of the j th iteration of loop, we assume $A[1 \dots j - 1]$ is sorted and then prove $A[1 \dots j]$ is sorted after this iteration.
- The body of the while loop (lines 4-7) works by moving $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, ... by one position to the right until the proper position for the key $A[j]$ is found.
- We insert the key at the point when $A[i] < key < A[i + 2]$.
- Therefore, $A[1 \dots j]$ is sorted before next iteration, which shows that the **loop invariant** holds at the start of the $(j + 1)$ th iteration of loop.



Termination (终止)

- For insertion sort, the outer for loop ends when j exceeds n , i.e. when $j = n + 1$.
- By loop invariant of maintenance step, we have that the subarray $A[1...n]$ consists of the elements originally in $A[1...n]$, but in sorted order.
- The subarray $A[1...n]$ is the entire array! Hence, the entire array is sorted, which means that **loop invariant** holds at every iteration and the algorithm is correct.

数学归纳法通常针对无穷序列, 但是我们的循环有终止, 还多了一个终止步.



EFFICIENCY OF ALGORITHM

- Is correctness of an algorithm enough? How good is an algorithm? 一个算法仅仅是正确的是不够的，还需要够好
- Time complexity (时间复杂度)
 - Indicates how fast an algorithm runs.
 - How many CPU cycles needed.
- Space complexity (空间复杂度)
 - Amount of memory units required by an algorithm.
- Does there exist a better algorithm?
- How to compare algorithms?



TIME COMPLEXITY (时间复杂度)

- Running time of an algorithm on a particular input generally be the number of primitive operations or "steps" executed. (算法执行基本运算 (或步数) 的数目)
- Define the notion of step so that it is as **machine independent** as possible. (比较与具体的计算机配置无关)
 - 一个算法在一台古老的Intel 586 CPU 上的运行时间和在i10 CPU上的运行时间比较是没有意义的
- The comparison should be **instance independent**. (比较与具体的数据输入无关)
 - 我们不是比较算法在某个特定输入数据的情况，而是比较一个问题的**通用**情况



TIME COMPLEXITY (时间复杂度)

- Consider insertion sort, let $A = \langle 1, 2, 3, 4, \dots, n \rangle$ and consider all $n!$ permutations of the elements in A . Each permutation corresponds to one possible input. (A中元素的所有 $n!$ 个排列, 每个排列对应于一个可能的输入)
- Consider three permutations:
 - Case 1: The elements in A are sorted in decreasing order.
 - Case 2: The elements in A are sorted in increasing order.
 - Case 3: The elements in A are randomly ordered.
- Will insertion sort have the same running time on these three cases?



TIME COMPLEXITY (时间复杂度)

度量一个算法的运行时间，有以下三种方式：

- The **worst-case time complexity (最坏情形时间复杂度)** of an algorithm is the function defined by the maximum number of steps taken on any instance of size n . (任何大小为 n 的实例上所需最多基本运算次数)
- The **best-case time complexity (最好情形时间复杂度)** of an algorithm is the function defined by the minimum number of steps taken on any instance of size n . (任何大小为 n 的实例上所需最少基本运算次数)
- The **average-case time complexity (平均情形时间复杂度)** of an algorithm is the function defined by an average number of steps taken on any instance of size n . (任何大小为 n 的实例上，任一实例，平均所需基本运算次数)
-
- Which of these is the best to use?



TIME COMPLEXITY (时间复杂度)

We use the following convention to analyze an algorithm:

- A **constant amount of time** is required to execute each line (operation) of our pseudocode. (常数时间)
- Each line may take a different amount of time than another line, so we shall assume that each execution of the **i th line** takes time **c_i** , which is a **constant**. (不同行常数时间不同)
- For a loop, we calculate how many **times the loop iterates** and then multiply with the total time within this loop.



分析插入排序时间复杂度

InsertSort(A)	Cost	Times
1 for $j \leftarrow 2$ to n do	c_1	n
2 $key \leftarrow A[j]$	c_2	$n - 1$
3 $i \leftarrow j - 1$	c_3	$n - 1$
4 while $i > 0$ and $A[i] > key$ do	c_4	$\sum_{j=2}^n t_j$
5 $A[i + 1] \leftarrow A[i]$	c_5	$\sum_{j=2}^n (t_j - 1)$
6 $i \leftarrow i - 1$	c_6	$\sum_{j=2}^n (t_j - 1)$
7 $A[i + 1] \leftarrow key$	c_7	$n - 1$
8 return A	c_8	1

- In each for loop, we run different times of while loop. Therefore, we use t_j as the number of condition tested of the j th while loop.

令 t_j 表示在**for**循环的迭代 j 中, **while**循环执行的次数。



分析插入排序时间复杂度

- Generally, the running time increases with n .
 - Sorting 5 numbers is obviously faster than sorting 10,000 numbers.
- Therefore, it is natural to represent the total running time $T(n)$ as a function of n :

$$\begin{aligned} T(n) = & c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n t_j \\ & + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7(n-1) + c_8 \end{aligned}$$



BEST CASE OF INSERTION SORT

对于给定大小的输入，算法的运行时间可能取决于问题的输入

- For inputs of a given size, an algorithm's running time may depend on which input of problem.
- **The best case** occurs for insertion sort if the array is **already sorted**. In this case:
 - The while loop condition fails in the beginning, i.e. $t_j = 1$ for all j .
 - Line 5-6 in the while loop are not executed.
- Then, $T(n)$ becomes:
$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) + c_8 \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7 - c_8) \end{aligned}$$
- This running time is a linear function of n .

InsertSort(A)算法的运行时间 $T(n)$ 可以表示为关于问题规模 n 的线性函数。



WORST CASE OF INSERTION SORT

- **The worst case** occurs if the array is in **reverse sorted order**. In this case:
 - The key is always moving to the front in the while loop and thus $t_j = j$.
- Then, $T(n)$ becomes:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n j \\ &\quad + c_5 \sum_{j=2}^n (j-1) + c_6 \sum_{j=2}^n (j-1) + c_7(n-1) + c_8 \\ &= \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2} \right) n^2 + \left(c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7 \right) n \\ &\quad - (c_2 + c_3 + c_4 + c_7 - c_8) \end{aligned}$$

- This running time is a quadratic function of n .

InsertSort(A)算法的运行时间 $T(n)$ 可以表示为关于问题规模 n 的二次函数



AVERAGE CASE OF INSERTION SORT

- **The average-case** or expected running time is taken to be the average time over all inputs of size n . (为大小为 n 的所有输入的平均时间)
- The average-case analysis needs know **the probabilities of all input occurrences**, i.e., it requires prior knowledge of the input distribution.
- The analysis is complex and lengthy in many cases.
- We will discuss it with probabilistic analysis in Lecture 3.

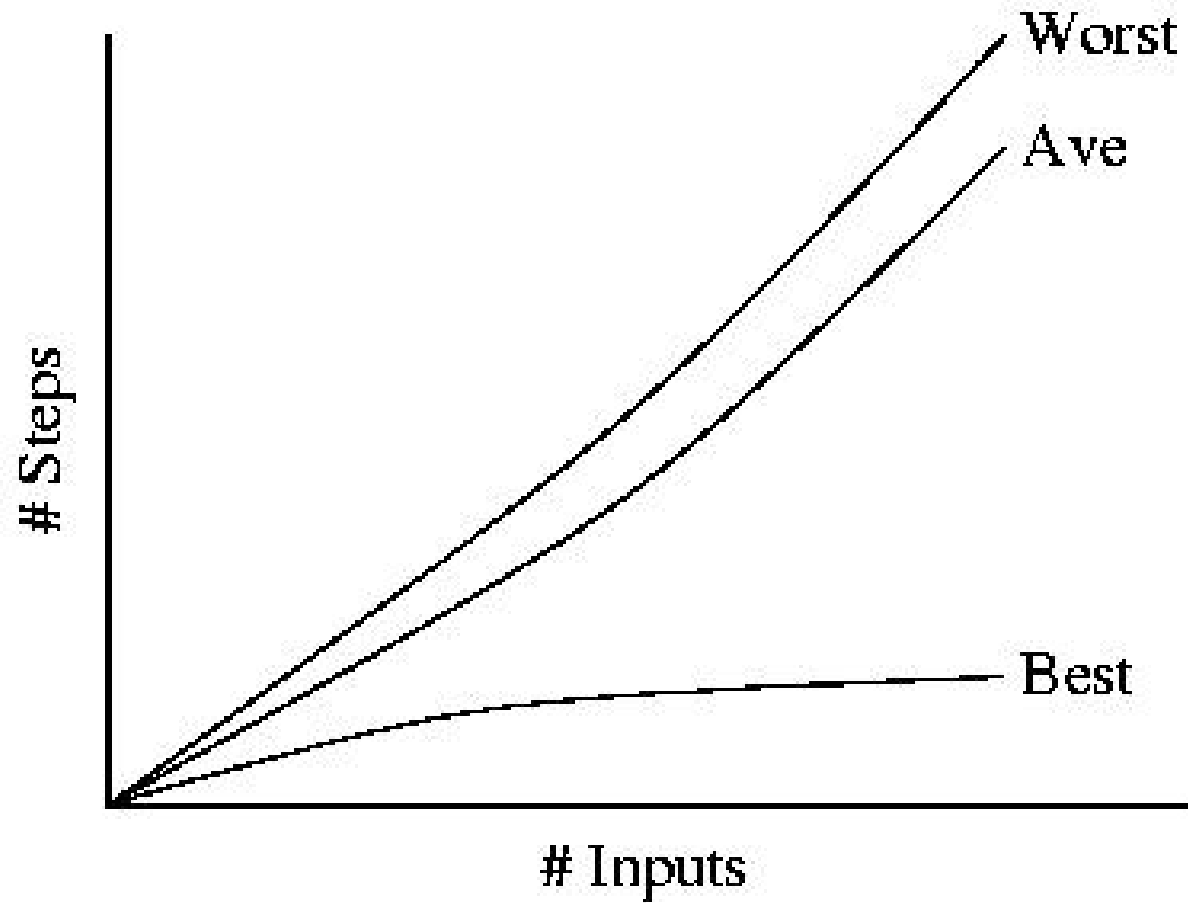


BEST-, WORST-, AND AVERAGE-CASE

- Best-case analysis is **not used in practice**, as it does not give useful information about the behavior of an algorithm in general.
 - We will not bet our algorithm on luck.
- Generally, we **concentrate on the worst-case** analysis.
 - The worst-case running time of an algorithm is an **upper bound** on the running time for **any** input instance. (实例运行时间的上界)
 - For some algorithms, the worst case **occurs fairly often**.
 - The average-case is often roughly as bad as the worst case.



BEST-, WORST-, AND AVERAGE-CASE





时间复杂度的比较

- Given two algorithms, we obtain two $T(n)$ and each of them has a lot of c_i . How to **compare them?**
- Consider the best-case of insertion sort $T(n) = cn - b$, where $c = c_1 + c_2 + c_3 + c_4 + c_7$ and $b = c_2 + c_3 + c_4 + c_7 - c_8$ are constants.
- As the problem size n increases, the running time is dominated by n and does not matter much by constants c_i .

随着问题大小 n 的增加，运行时间由 n 决定，而常数 c_i 并不重要。



时间复杂度的比较

- Consider two sorting algorithms:
 - **Insertion sort** takes time roughly equal to $c_1 n^2$ to sort n items.
 - **Mergesort** takes time roughly equal to $c_2 n \lg n$ to sort n items.
- Assume Mergesort has larger constant: $c_1 = 1$ and $c_2 = 100$.
- For a small problem with $n = 10$:
 - Insertion sort takes $1 \times 10^2 = 100$ operations.
 - Mergesort takes $100 \times 10 \lg 10 \approx 3322$ operations.
- However, for a big problem with $n = 10^6$:
 - Insertion sort takes $1 \times 10^{12} = 10^{12}$ operations.
 - Mergesort takes $100 \times 10^6 \lg 10^6 \approx 1.99 \times 10^9$ operations.



时间复杂度的比较

- In the theoretical analysis of an algorithm, we are interested in the **eventual behavior**. 我们关心的是当问题规模 n 足够大的时候，算法的复杂度
 - We compare algorithms for **sufficiently large n** .
- We can show that, no matter how large c_2 is, we can always find a $n > n_0$ such that $c_1 n^2 > c_2 n \lg n$.
- This is because the term n^2 has **higher order (高阶)** than $n \lg n$.
- Therefore, when we compare the efficiency of algorithms, we concentrate on the order and ignore constants.

我们比较算法的效率时，我们专注于阶数而忽略常数。



ASYMPTOTIC RUNNING TIME

增长率或增长量级，只考虑公式中的最高项，并且忽略系数

- We are actually interested in the rate of growth, or order of growth. Intuitively, just look at the dominant term.

$$T(n) = \cancel{0.1n^3} + \cancel{10n^2} + \cancel{5n} + \cancel{25}$$

- Drop lower-order terms $10n^2 + 5n + 25$.
- Ignore constant 0.1.
- But we can't say that $T(n)$ equals to n^3 .
 - It grows like n^3 . But it doesn't equal to n^3 .
- We will define asymptotic notations (渐进符号) in Lecture 2 to formally describe the time complexity of an algorithm.



LOWER BOUND (下界) OF A PROBLEM

问题的下界，即任何一种算法解决一个问题所必需的最小运行时间

- Given a problem, how can we know if we still have chance to improve an algorithm?
- The lower bound of a problem characterizes the **complexity of a problem**, informally as “the best you can do.” Namely, the minimum time needed by **any algorithm** to solve the problem.
- If we have an algorithm whose **worst-case is equal to the lower bound of the problem**, we can claim that this algorithm is optimal to the problem and the problem is solved.
 - Then, we don't have to waste time on this problem and focus on other interesting problems.

一个算法其最坏情况等于问题的下界，我们可以声称该算法对问题是最优的。



FIND THE MAXIMUM VALUE

FindMax()

```
1 max ← A[1]
2 for  $j \leftarrow 2$  to  $n$  do
3   if  $A[j] > \text{max}$  then
4     max ←  $A[j]$ 
5 return max
```

- Lower bound of the problem is $n - 1$.
- The worst-case of this algorithm is $n - 1$.



CONCLUSION

After this lecture, you should know:

- What is a problem and an algorithm?
- Why do we need to study algorithm?
- How to write pseudocode for an algorithm?
- Why do we need to study the efficiency of an algorithm?
- What are the three cases of time complexity?
- What is the lower bound of a problem?



HOMework (课本习题)

- Page 11-12

1.2 1.5

1.6 1.7

1.8 1.9

1.10 1.11

下周一之前交



EXPERIMENT (编程作业)

这次不用交编程作业



有问题欢迎随时跟我讨论

曹刘娟

caoliujuan@xmu.edu.cn